

Data Analysis with PythonNumpy

Numpy is a fundamental library for Scientific computing in Python. It provides support for arrays and matrices, along with a collection of mathematical functions to operate on these data structures.

→ # Install the library

!pip install numpy

→ # Load the library

import numpy as np

→ # Create Array Using numpy

Create a 1D Array

arr1 = np.array([1, 2, 3, 4, 5])

print(arr1)

[1 2 3 4 5]

print(type(arr1))

<class 'numpy.ndarray'>

print(arr1.shape)

(5,)

→ reshape into 2 dimensional array

arr2 = np.array([1, 2, 3, 4, 5])

arr2.reshape(1, 5) # 1 row and 5 columns

array([[1, 2, 3, 4, 5]])

Manual Create

arr2 = np.array([[1, 2, 3, 4, 5]])

arr2.shape

(1, 5)

2D Array

→ `arr2 = np.array([[1,2,3,4,5],[2,3,4,5,6]])`

`print(arr2)`

`[[1 2 3 4 5]`

`[2 3 4 5 6]]`

2 dimensions

`[[[[` ← 3 dimensions

`print(arr2.shape)`

`(2,5)`

included

Excluded

→ `np.arange(0,10,2)`

0 to 10, 2 by 2 increment

`array([0, 2, 4, 6, 8])`

Convert to a 2D

`np.arange(0,10,2).reshape(5,1)`

`array([0],`

`[2],`

`[4],`

`[6],`

`[8]])`

→ `np.ones(3,4)`

`array([[1., 1., 1., 1.],`

`[1., 1., 1., 1.],`

`[1., 1., 1., 1.]])`

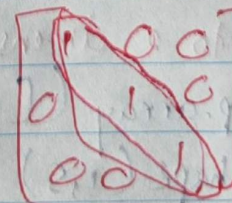
Identity Arrays

`np.eye(3)`

`array([[1., 0., 0.]`

`[0., 1., 0.]`

`[0., 0., 1.]])`



Attributes of Numpy Array

```

→ arr = np.array([[1,2,3],[4,5,6]])
print("Array:\n", arr)
print("Shape", arr.shape) # Output: (2,3)
print("Number of dimensions:", arr.ndim) # Output: 2
print("Size (number of elements):", arr.size) # Output: 6
print("Data type:", arr.dtype) # Output: int64
print("Item Size (in bytes):", arr.itemsize) # Output: 8

```

→ # Numpy Vectorized Operation

```

arr1 = np.array([1,2,3,4,5])
arr2 = np.array([10,20,30,40,50])
# Element wise Addition
print("Addition:", arr1 + arr2) [11, 22, 33, 44, 55]
# Element wise Subtraction
print("Subtraction:", arr1 - arr2) [-9, -18, -27, -36, -45]
# Element wise Multiplication
print("Multiplication:", arr1 * arr2) [10, 40, 90, 160, 250]
# Element wise Division
print("Division:", arr1 / arr2) [0.1, 0.1, 0.1, 0.1, 0.1]

```

→ # Universal Functions

```

arr = np.array([1, 2, 3, 4, 5])
# Square root
print(np.sqrt(arr)) [1.4142, 1.7320, 2, 2.2360, 2.4494]
# Exponential
print(np.exp(arr)) [7.38, 20.08, 54.59, 148.41, 403.42]
# Sine
print(np.sin(arr)) [0.9092, 0.1411, -0.7568, -0.9589, -0.2794]

```


natural log $\log([2, 2, 4], [8, 8, 1])$ $\log(2) = 0.6931$
 Print (np.log(arr)) [0.6931, 1.0986, 1.3862, 1.6094, 1.7918]

→ Universal Functions (ufuncs)

Array Slicing and Indexing

```
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
print("Array:\n", arr)
[[1 2 3 4]
 [5 6 7 8]
 [9 10 11 12]]
```

```
print(arr[0])
```

```
array([1, 2, 3, 4])
```

```
print(arr[0][0])
```

```
1
```

```
print(arr[2:3])
```

```
print(arr[1:, 2:])
```

```
array([[7, 8],
```

```
[11, 12]])
```

```
print(arr[0:2][2:])
```

```
[[3 4]
 [7 8]]
```

0	1	2	3	4
5	6	7	8	
9	10	11	12	

0	1	2	3	4
5	6	7	8	
9	10	11	12	

→ # Modify Array Element

```
arr[0,0] = 100
```

```
print(arr)
```

```
arr[1:] = 100
```

```
print(arr)
```

100	2	3	4
100	100	100	100
100	100	100	100

→ # Statistical Concept -- Normalization

To have a mean of 0 and Standard deviation of 1

```
data = np.array([1,2,3,4,5])
```

Calculate the mean and Standard deviation

```
mean = np.mean(data)
```

```
std_dev = np.std(data)
```

```
Variance = np.var(data)
```

Normalize the data

```
normalized_data = (data - mean) / std_dev
```

```
print("Normalized data:", normalized_data)
```

Output

Normalized data: $[-1.2649, -0.6324, 0, 0.6324, 1.2649]$

→ # Logical Operations

```
data = np.array([1,2,3,4,5,6,7,8,9,10])
```

```
data > 5
```

```
[False, False, False, False, False, True, True, True, True, True]
```

```
data[data > 5]
```

```
[6, 7, 8, 9, 10]
```

```
data[(data > 5) and (data <= 8)]
```

```
[5, 6, 7, 8]
```